

塔克串口示波器

使用手册

V2.3



更多资讯 · 欢迎关注塔克创新微信公众号

烟台塔克电子科技有限公司

版权申明

本手册版权归属塔克创新所有，并保留一切权力，受法律保护。非经(书面形式)同意，任何单位及个人不得擅自摘录或修改本手册部分或全部内容，违者我们将追究其法律责任。

塔克媒体

塔克官网	www.xtark.cn
淘宝店铺	https://xtark.taobao.com
塔克哔哩	https://space.bilibili.com/511052131
销售邮箱	sales@xtark.cn

塔克淘宝，微信公众号二维码



更多产品 · 欢迎关注塔克创新淘宝店铺



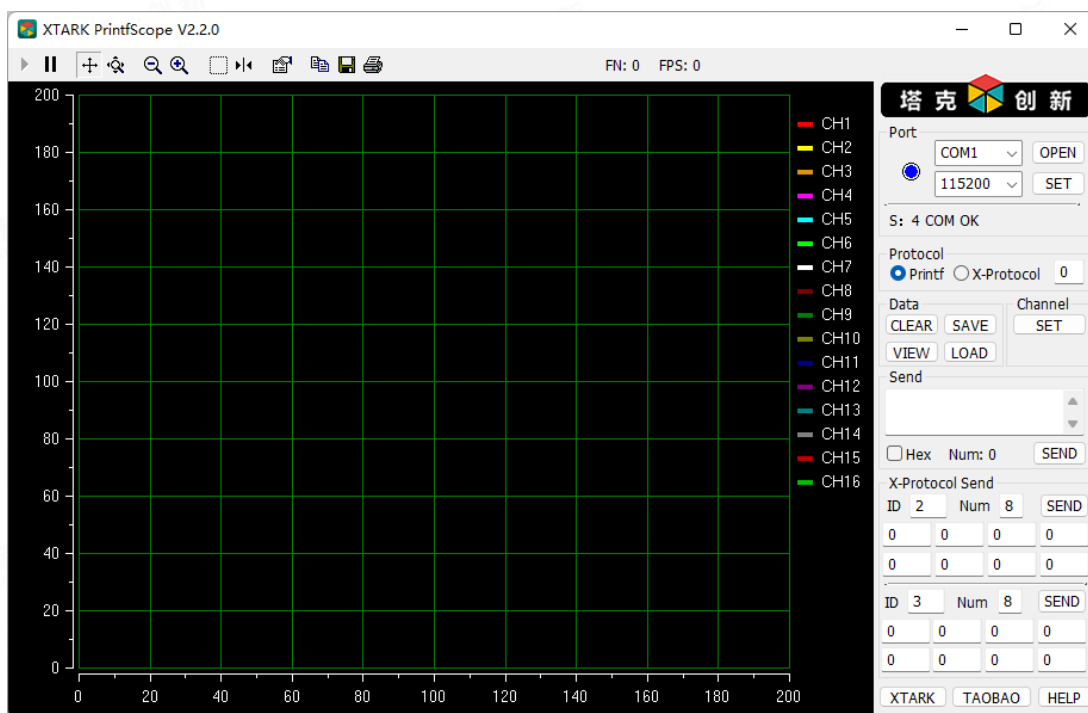
更多资讯 · 欢迎关注塔克创新微信公众号

本手册为塔克创新自主研发的串口示波器软件使用说明手册，软件不断更新，最新软件和手册请关注“塔克创新”公众号获取。

1. 产品介绍

塔克 PrintfScope 串口示波器，如其名，软件最大亮点是可以通过 C 语言 Printf 函数实现波形显示功能，使用简单灵活。另外也支持塔克创新通用串口 X-Protocol 通信协议，实现高效率传输。

PrintfScope 串口示波器软件具有 16 个显示通道，最大特点是可通过 C 标准库函数 printf 增加 @ 标识实现数据波形显示，例如 `printf( "@%d %f ", a, b)` 可实现一个整形一个浮点类型数据波形显示，使用方便灵活。Printf 协议使用字符编码方式，虽然使用方便灵活，但传输效率相对较低，本软件也支持高效的塔克通用 X-Protocol 协议传输方式。



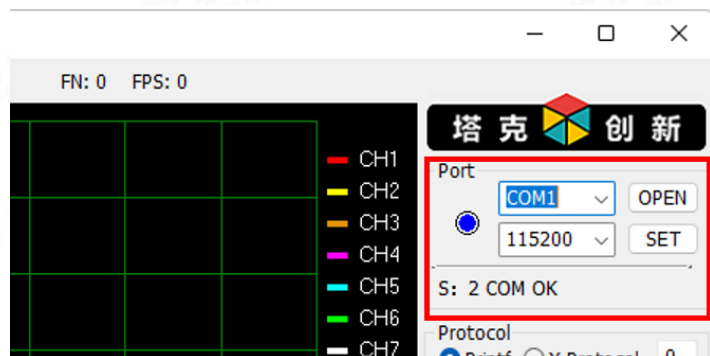
PrintfScope 基于功能强大的 locomp 控件设计，软件界面及功能介绍如下图所示，详细使用说明见下文。



2. 使用说明

2.1. 串口设置

使用软件前，首先设置串口，具体如下图所示。



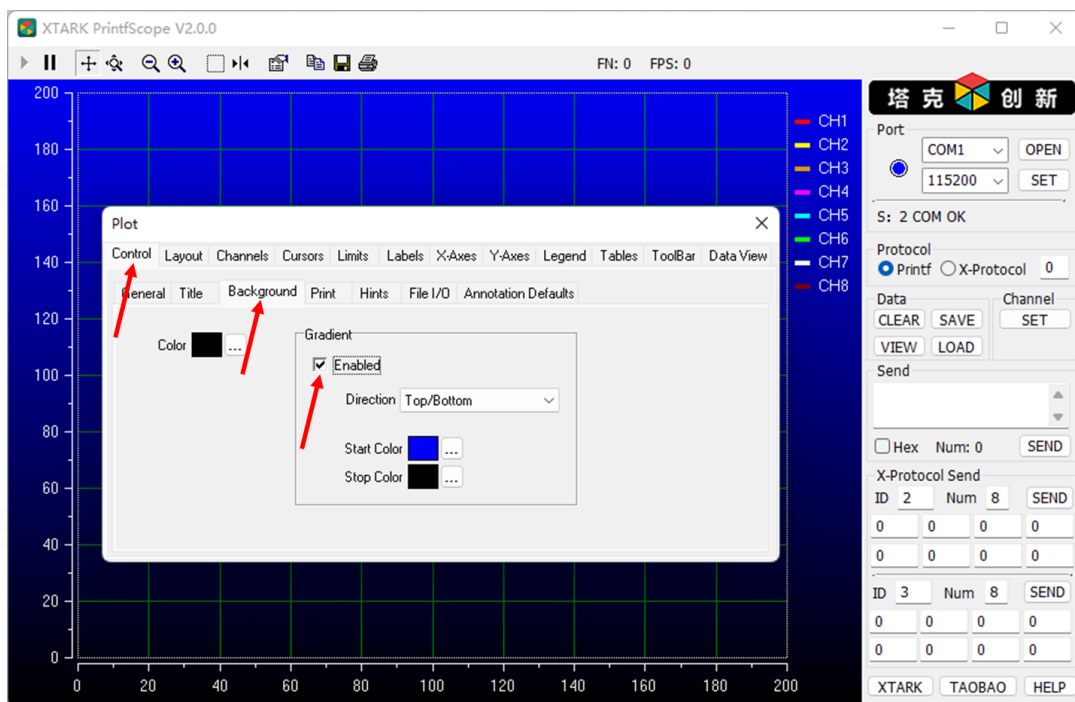
- 软件启动后，会自动搜索可用的串口，并在端口号下拉框处显示可用串口。如果有可用串口单击“打开串口”按钮即可打开对应串口，并且显示串口状态和参数信息（S:COM3_ON 115200 N 8 1），串口参数为默认参数（波特率：115200，校验位：NONE，数据位：8，停止位1）。
- 端口号下拉框可选择和切换不同串口。
- 波特率下拉框可以选择不同波特率。
- 设置串口按钮可以打开不常用的“校验位”“数据位”“停止位”串口参数设置对话框。
- 进入串口参数设置对话框可进行参数设置，完成后单击参数更新即可完成参数设置。考虑到常用情况，参数设置不具有记忆性，每次进入均显示常用的默认参数值。
- 不支持串口热插拔，热插拔会报错误，建议关闭串口后再进行串口插拔。

2.2. 波形显示

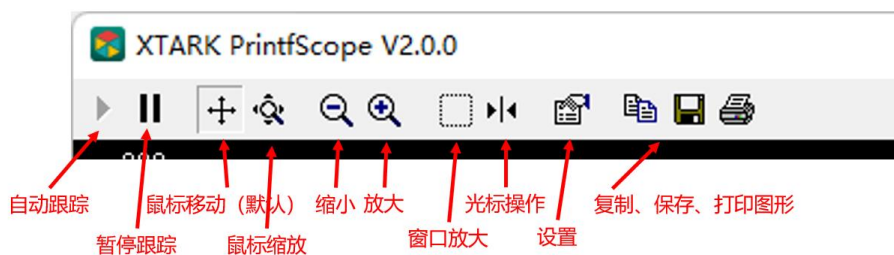


- 具有 16 个波形显示通道，使用不同颜色表示
- 波形可以拖动、放大、缩小操作
- 示波器上方具有帧数和帧率显示
- CLEAR 按钮实现波形和数据清除
- VIEW 按钮实现数据显示（待实现）
- SAVE 按钮实现数据保存为 txt 格式文件，文件名默认为“xx 月 xx 日 xx 时 xx 分 xx 秒 .txt”，其中 xx 为当前时间，“_”后可增加内容字符描述。
- LOAD 按钮可实现保存数据的波形显示（待实现）
- Axes 栏 X、Y 滑动条可进行 X 轴和 Y 轴数据缩放操作

X-PrintfScope 基于功能强大的 locomp 控件设计，**locomp 控件功能和设置项非常多**，更多功能请在波形显示窗口右键进入设置项探索。如下介绍一个更换背景色功能。



波形操作，控制快捷键介绍如下。



具有帧 ID 筛选功能，解决多种帧数据发送的场景下无法显示波形的问题，通过帧 ID 筛选，可以实现特定帧 ID 的波形显示，例如只显示帧 ID 为 1 的数据。默认为 0，不判断帧 ID，所有 ID 数据均显示，当修改为特定帧 ID 时，则只显示该 ID 帧数据。



放大至全屏显示效果非常震撼，同样需要的显示资源较多，频率较快时容易出现延时。



注意：波形显示帧率与通道数量，窗口大小，电脑显示性能有关，包括显卡和 CPU 性能，帧率过大容易导致波形显示延迟。窗口越大，需要显示资源越多，帧率会较慢，建议调低发送频率，建议通道较多时最优发送帧率为 50，通道少增加至 100。

2.3. 通道操作

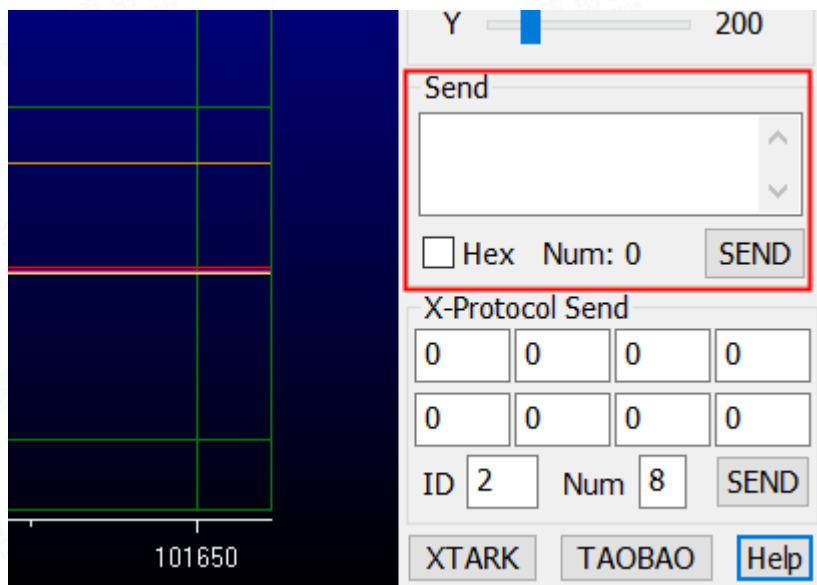
Channel Setting ✕

	Name	View		Scale		Shift
CH1:	加速度X	☒	OUTPUT = INPUT *	1.00	+	0.00
CH2:	加速度Y	☒	OUTPUT = INPUT *	1.00	+	0.00
CH3:	加速度Z	☒	OUTPUT = INPUT *	1.00	+	0.00
CH4:	陀螺仪X	☒	OUTPUT = INPUT *	1.00	+	0.00
CH5:	CH5	☒	OUTPUT = INPUT *	1.00	+	0.00
CH6:	CH6	☒	OUTPUT = INPUT *	1.00	+	0.00
CH7:	CH7	☒	OUTPUT = INPUT *	1.00	+	0.00
CH8:	CH8	☒	OUTPUT = INPUT *	1.00	+	0.00

通道命名
显示开关
比例变化
偏移变化

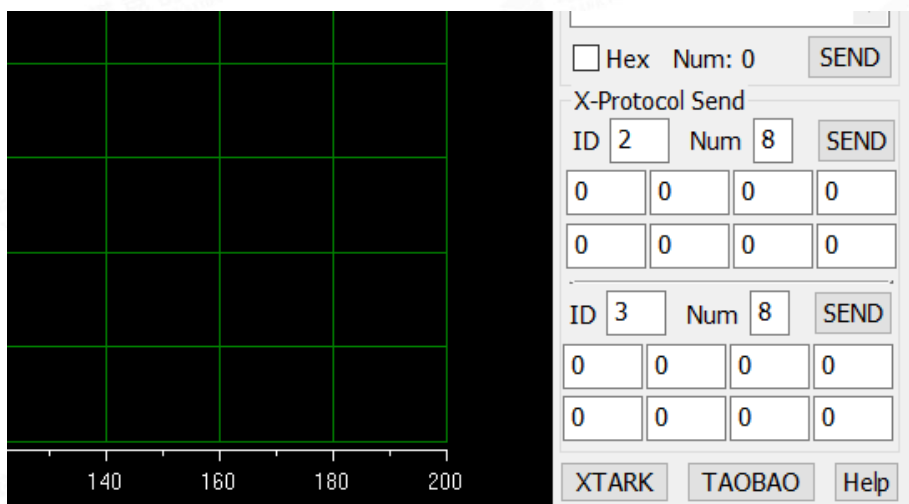
- Channel 栏 SET 按钮可实现波形通道相关操作，单击可进入通道设置对话框
- 通道设置对话框 Name 栏可定义波形通道名称
- View 复选框可控制该通道波形是否显示
- Scale 栏为输出与输入的比例系数
- Shift 栏为输出与输入的唯一系数
- Scale 和 Shift 可以方便进行波形显示范围的控制，方便进行不同数量级波形同屏显示

2.4. 普通发送



- 为方便系统调试，设计了数据发送功能，可实现下位机命令交互功能。
- 可以在 Send 区，发送您发送的任意字符。
- 支持 16 进制发送，勾选 'Hex' 复选框后，发送的时候将对发送区的内容进行 16 进制和字符互转。
- Num 为发送的字节数。

2.5. 协议发送



- 支持 2 帧数据定义。
- 实现 8 个有符号 16 位数据的发送功能，数据范围 - 32767~32767，主要用于命令控制和参数调试，例如 PID 参数调试。
- 数据传输协议，采用 X - Protocol 协议

- 帧长度：根据传输数据 NUM 的数量自动计算
- ID-帧编码：可用户自定义，默认 0x02，范围 0~255
- 数据：数据内容为根据用户需要填写，范围 - 32767~32767, 16 位有符号型，根据前面定义数据数量发送，例如数量为 2，则发送数据 1 和数据 2。
- 校验和：前面数据累加和的低 8 位

3. 协议说明

3.1. 协议介绍

Printf 协议说明

- Printf 协议：通过 C 语言 Printf 函数增加@标识实现数据波形显示。
- 显示示例函数：printf("@%d %d %f %f \r\n", a, b, x, y); , 实际发送数据 @876 142 12.35 48.357

- @为帧头标识符，回车换行\r\n为帧尾标识符。
- @帧头后面紧跟数据。
- 每个数据后面加一个空格，作为数据标识符。
- 支持整型%d 和浮点%f 两种数据类型。
- 数据数量为 1~8，几个数据对应几个显示通道

X-Protocol 协议说明

为了更好的使用串口进行数据传输，塔克创新设计了一个通用的通信传输协议 X-Protocol 协议，方便塔克创新产品使用，包括软件产品和硬件产品。

X-SOFT通用串口传输协议：X-Protocol协议（变帧长）

AA 55 0B 01 00 01 00 02 00 03 55

帧头 帧长度 帧号 数据：高位在前，长度可变，自由组合 校验和

帧头	双帧头，抗干扰强
帧长度	根据数据长度设定
帧号	用户根据功能设定，标识帧的唯一性
数据	高位在前，长度可变，内容自由组合8位，16位，32位数据
校验和	前面数据累加和的低8位

3.2. 参考代码

使用 OpenCTR 控制器的参考接收代码。

```
//接收
static u8 UART_RX_BUF[40]; //接收缓冲，数据内容小于等于32Byte
static u8 UART_RX_CON = 0; //接收计数器
static u8 UART_RX_CHECKSUM; //帧头部分校验和
/**
 * @简 述  串口中断服务程序
 * @参 数  无
 * @返回值  无
 */
void USART1_IRQHandler(void)
{
    uint8_t Res;

    if (USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) //接收中断
    {
        Res = USART_ReceiveData(USART1);

        if (UART_RX_CON < 3) //==接收帧头 + 长度
        {
            if (UART_RX_CON == 0) //接收帧头1 0xAA
            {
                if (Res == 0xAA)
                {
                    UART_RX_BUF[0] = Res;
                    UART_RX_CON = 1;
                }
                else
                {
                }
            }
        }
        else if (UART_RX_CON == 1) //接收帧头2 0x55
        {
            if (Res == 0x55)
            {
                UART_RX_BUF[1] = Res;
                UART_RX_CON = 2;
            }
            else
            {
                UART_RX_CON = 0;
            }
        }
    }
}
```

```
    }  
  }  
  else //接收数据长度  
  {  
    //USART1_RX_LEN = ( Res-1 );  
    UART_RX_BUF[2] = Res;  
    UART_RX_CON = 3;  
    UART_RX_CHECKSUM = 0xFF + Res; //计算校验和, 0xFF为0xAA、0x55校验和  
  }  
}  
else //==接收数据  
{  
  if (UART_RX_CON < (UART_RX_BUF[2] - 1))  
  {  
    UART_RX_BUF[UART_RX_CON] = Res;  
    UART_RX_CON++;  
    UART_RX_CHECKSUM = UART_RX_CHECKSUM + Res;  
  }  
  else //判断最后1位  
  {  
    //由于没有配套软件, 暂时无需校验  
    if (Res == UART_RX_CHECKSUM) //校验正确  
    {  
      //此处进行数据解析  
      UART_Unpack();  
    }  
    else //校验错误  
    {  
      //不处理, 输出错误提示  
    }  
  }  
  
  //接收完成, 恢复初始状态  
  UART_RX_CON = 0;  
}  
}  
USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);  
}
```

4. 其它说明

暂无。